

Reinforcement Learning on CliffWalking-v1:

A Comparative Study of Tabular Q-Learning and Deep Q-Networks

Bogdan Andrei Tutuianu
2217481

September 2026

Table of Contents

1. Introduction	3
2. Theoretical Background	3
2.1 Markov Decision Processes	4
2.2 The Q-function and the Bellman equation	4
2.3 Non-deterministic Q-learning (tabular case)	4
2.4 From tables to function approximation: motivation for DQN	5
2.5 Deep Q-Network (DQN)	5
2.6 Evaluating a Reinforcement Learning agent	6
3. Environment and Common Methodology	6
3.1 The CliffWalking-v1 environment	6
3.2 Evaluation protocol	7
3.3 Experimental methodology	7
3.4 Shared implementation infrastructure	8
4. Tabular Q-Learning	9
4.1 Implementation	9
4.2 Deterministic environment: validating the update rule	9
4.3 Stochastic baseline	10
4.4 Hyperparameter isolation	11
4.5 Combined configurations and final choice	13
4.6 Summary	14
5. Deep Q-Network (DQN)	15
5.1 Implementation	15
5.2 Deterministic environment	16
5.3 Stochastic baseline	16
5.4 Hyperparameter isolation	17
5.5 Training loss	19
5.6 Final configuration	20
5.7 Summary	20
6. Comparative Analysis: Tabular Q-Learning vs DQN	21
6.1 Evaluation return	22
6.2 Training time	23
6.3 Stability and failure modes	23
6.4 Sample efficiency	24
6.5 Summary of the comparison	24
7. Conclusion	24
7.1 Summary of findings	24
7.2 Practical takeaways	25

1. Introduction

Reinforcement Learning (RL) addresses the problem of learning an optimal behaviour for an agent that interacts with an unknown environment, receiving a scalar reward signal after each action and using it to improve its decisions over time. Unlike supervised learning, no labelled examples of the optimal policy are available: the agent must discover good behaviour purely from trial and error, choosing actions, observing the resulting state, and collecting a reward.

This report addresses the CliffWalking-v1 environment from the Gymnasium library, a classic gridworld benchmark used to study the trade-off between exploration and safe, reward-maximizing behaviour. Two independent learning agents are implemented from scratch and compared:

- Tabular Q-Learning, which represents the action-value function $Q(s,a)$ explicitly as a table and updates it with the non-deterministic Bellman update rule.
- Deep Q-Network (DQN), which replaces the table with a neural network $Q_\theta(s,a)$ trained by gradient descent on the same Bellman target, using experience replay to decorrelate training samples.

Both agents are trained and evaluated under the same protocol, first in a deterministic version of the environment (used to validate the correctness of the update rule against the known optimal return) and then in the stochastic version required by the project guidelines, where the outcome of an action is only partially controlled by the agent. For each agent, a structured set of experiments isolates the effect of individual hyperparameters (learning rate, discount factor, exploration schedule, and — for DQN — network width, batch size and replay-buffer size) before combining the best settings into a final configuration.

The remainder of the report is organised as follows. Section 2 reviews the theoretical background used throughout the project, in particular the non-deterministic Bellman update rule and the function-approximation view of Q-learning that motivates DQN. Section 3 describes the CliffWalking-v1 environment and the experimental methodology shared by both agents. Sections 4 and 5 present the implementation, experiments and results for Tabular Q-Learning and DQN respectively. Section 6 compares the two agents directly, and Section 7 summarises the main findings and limitations of the work.

2. Theoretical Background

This section summarises the theoretical tools used in the rest of the report, following the notation introduced in the course (Patrizi, “MDPs and Reinforcement Learning”). All formulas are reproduced in the exact form used to derive the two implementations, since they are the basis for the update rules coded in `tabular_q_learning.py` and `dqn_cliffwalking.py`.

2.1 Markov Decision Processes

The environment is modelled as a Markov Decision Process (MDP), a tuple $M = (S, A, \delta, r)$ where:

- S is a finite set of states,
- A is a finite set of actions,
- $\delta(s' | s, a)$ is a probability distribution over the next state, given the current state and action (the transition model),
- $r : S \times A \times S \rightarrow \mathbb{R}$ is the reward function.

An MDP satisfies the Markov property: the next state depends only on the current state and the executed action, not on the history that led to it. CliffWalking-v1 is naturally modelled this way: the state is the agent's grid cell, and the action is one of the four cardinal moves.

A policy $\pi : S \rightarrow A$ selects an action for every state. The quality of a policy is measured by its expected cumulative discounted reward from a state s :

$$V^\pi(s) \equiv E[r_1 + \gamma r_2 + \gamma^2 r_3 + \dots], \gamma \in [0, 1)$$

where γ is the discount factor that trades off immediate versus future reward, and V is called the value function. An optimal policy π^* satisfies $V^{\pi^*}(s) \geq V^\pi(s)$ for every state s and every other policy π .

2.2 The Q-function and the Bellman equation

Explicitly maximising over policies is not practical, so RL algorithms instead learn the action-value function $Q(s,a)$: the expected return obtained by executing action a in state s and then following the optimal policy thereafter. In the deterministic case:

$$Q(s, a) \equiv r(s, a) + \gamma \cdot V^*(\delta(s, a))$$

Since $V^*(s) = \max_a Q(s,a)$, the same quantity can be written self-referentially as a fixed-point equation (the Bellman optimality equation):

$$Q(s, a) = r(s, a) + \gamma \cdot \max_{a'} Q(\delta(s, a), a')$$

Once Q is known, the optimal policy is simply $\pi^*(s) = \operatorname{argmax}_a Q(s,a)$: no model of δ or r is required at decision time. This is the key property that both agents implemented in this project exploit.

2.3 Non-deterministic Q-learning (tabular case)

CliffWalking-v1 is used in its stochastic form (the `is_slippery` flag), so the transition function δ is itself a probability distribution and the deterministic update above does not apply directly. The course generalises it to the non-deterministic training rule:

$$\hat{Q}_n(s, a) \leftarrow \hat{Q}_{n-1}(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} \hat{Q}_{n-1}(s', a') - \hat{Q}_{n-1}(s, a)]$$

with α a learning rate. This is exactly the update rule implemented in the `train()` function of `tabular_q_learning.py` (the code comment in the script references it as “slide 48”, i.e. this

formula). The course notes that the deterministic Q-learning rule does not converge in non-deterministic worlds, while this TD-style update converges as long as every state-action pair is visited infinitely often (Watkins and Dayan, 1992), with convergence also guaranteed for a fixed $\alpha \in [0, 1)$ — the choice made in this project, rather than the visit-count-based $\alpha = 1 / (1 + \text{visits}(s,a))$ also presented in the slides.

Action selection during training follows the ϵ -greedy strategy: with probability ϵ a random action is chosen (exploration), and with probability $1-\epsilon$ the greedy action $\text{argmax}_a Q(s,a)$ is chosen (exploitation). ϵ is linearly annealed from an initial value toward a small floor over the course of training, so that the agent explores broadly at first and exploits its learned policy later.

2.4 From tables to function approximation: motivation for DQN

An explicit Q-table is only feasible when the state-action space is small enough to visit every entry repeatedly. As the course notes explicitly, tabular methods perform “rote learning” with no generalisation across unseen state-action pairs, which becomes a bottleneck for larger or continuous state spaces. The standard remedy is function approximation: replacing the table $Q(s,a)$ with a parametric function $Q_\theta(s,a)$, for instance a linear combination of features or, as used here, a neural network. Learning then consists of adjusting θ (the network weights) so that Q_θ satisfies the same Bellman relation as the tabular Q — this is the Deep Q-Network (DQN) approach.

2.5 Deep Q-Network (DQN)

DQN keeps the same non-deterministic Bellman target used in tabular Q-learning, but computes it with a neural network instead of a table, and fits the network by minimising the squared error between its current prediction and that target. For a transition (s, a, r, s') :

$$y = r + \gamma \cdot \max_{a'} Q_\theta(s', a') \quad , \quad \mathcal{L}(\theta) = (Q_\theta(s, a) - y)^2$$

Two implementation choices, both mandated by the project guidelines, are worth making explicit here because they are natural questions for an oral discussion of how DQN works internally:

- Single Q-network: the project explicitly requires a single network, not the Double-DQN (D-DQN) architecture with a separate target network and policy network. The target y above is therefore computed with the very same network Q_θ that is being trained. To avoid the target “chasing itself” in an unstable feedback loop, the target is computed under `torch.no_grad()` so that gradients only flow through the predicted term $Q_\theta(s,a)$, not through the $\max_{a'} Q_\theta(s',a')$ term.
- Experience replay: transitions $(s, a, r, s', \text{done})$ are stored in a fixed-size buffer and minibatches are sampled uniformly at random for each gradient step, instead of training on consecutive transitions. This breaks the strong temporal correlation between consecutive samples and reuses past experience, which is what makes training a neural network on sequential RL data tractable.

The network used in this project takes a one-hot encoding of the discrete state as input (precomputed once as an identity matrix, since CliffWalking-v1 has only 48 states) and

outputs one Q-value per action, following the “network with s as input and one output per action” scheme mentioned in the course as the standard way to use a neural network as a Q approximator. Architecture and training details are given in Section 5.1.

The same ϵ -greedy exploration strategy and linear epsilon decay used for the tabular agent are reused for DQN, so that the only structural difference between the two agents is the representation of Q (table vs. network) and the consequent need for a replay buffer and minibatch gradient steps.

2.6 Evaluating a Reinforcement Learning agent

The course notes that the raw cumulative-reward curve is a noisy signal, because it mixes the outcome of exploratory (random) actions with the agent's actual learned behaviour, and suggests periodically evaluating the current greedy policy separately, with exploration switched off. This is the protocol adopted for both agents in this project: a fixed number of evaluation episodes are run after training, always with $\epsilon = 0$ (fully greedy action selection), and reported separately from the training-time metrics. Section 3.2 details the exact protocol used.

3. Environment and Common Methodology

3.1 The CliffWalking-v1 environment

CliffWalking-v1 is a 4×12 gridworld with 48 discrete states, numbered row-major from 0 (top-left) to 47 (bottom-right). The agent starts every episode in state 36 (bottom-left) and has to reach the goal in state 47 (bottom-right). Four actions are available in every state: Up, Right, Down and Left. The bottom row between the start and the goal, excluding the two endpoints, is the cliff.



Figure 3.1 – The CliffWalking-v1 grid: start (bottom-left), goal (bottom-right), cliff region (bottom row).

The reward structure is fixed by the environment:

- every step gives a reward of -1 , regardless of the action taken;

- stepping into the cliff gives a reward of -100 and immediately resets the agent to the start state, without ending the episode;
- reaching the goal state ends the episode.

Because every non-terminal step costs -1 , the optimal deterministic policy is the shortest path that hugs the top of the cliff without ever stepping into it: 13 steps from start to goal, for a return of -13 . This value is used throughout the report as the reference optimum and is plotted as a dashed horizontal line in every reward-per-episode figure.

The `is_slippery` flag controls whether the environment is deterministic or stochastic. With `is_slippery=False`, every action moves the agent exactly as intended. With `is_slippery=True`, the outcome of an action is only partially controlled: the agent moves in the intended direction with high probability, and there is a residual probability of ending up in a different adjacent cell. This makes the transition function δ a genuine probability distribution rather than a deterministic mapping, which is exactly the setting the non-deterministic Bellman update rule of Section 2.3 is designed for. The project guidelines require using the stochastic environment as the default setting for all experiments; the deterministic setting is used only as a sanity check, to confirm that both agents can recover the known optimal return of -13 before stochasticity is introduced.

3.2 Evaluation protocol

For every experiment, two distinct sets of metrics are recorded:

- Training-time metrics: episode reward, episode length, success flag and exploration rate ϵ , logged for every training episode. These curves include the effect of exploration, so they are noisy and are summarised as an average over the last 100 training episodes.
- Evaluation metrics: after training completes, the greedy policy ($\epsilon=0$, no exploration) is run for 100 separate episodes, and the average return and success rate over these episodes are reported as the final performance of the agent. Evaluation episodes are never used to update Q or θ , so this number reflects only what the agent learned, not the exploration noise present during training.

An episode counts as successful only if it terminates by reaching the goal state (state 47); an episode that reaches the maximum number of steps (`max_episode_steps`) without reaching the goal is counted as a failure, even if the agent fell into the cliff and respawned multiple times along the way. This choice matters because the stochastic environment can push the agent off its intended path: a strict success definition keeps the metric meaningful even when a run experiences several cliff falls before eventually reaching the goal.

3.3 Experimental methodology

Both agents are studied with the same four-phase experimental procedure, applied independently to Tabular Q-Learning (Section 4) and DQN (Section 5):

1. Baseline: a first configuration, based on the values suggested for the environment, is trained and evaluated to obtain a reference point.

2. Single-parameter isolation: starting from the baseline, one hyperparameter at a time is changed (learning rate, discount factor, number of episodes, exploration decay, and — for DQN — network width, batch size, replay-buffer size), keeping everything else fixed, to isolate its individual effect on training stability and final return.
3. Targeted combinations: configurations that combine two or more of the changes found to be beneficial (or interesting) in phase 2 are tested together, to check whether their effects compose as expected.
4. Final configuration: the best-performing setting found across the previous phases is selected and reported as the final result for that agent.

This is a guided search rather than an exhaustive grid search: the goal is to understand the effect of each parameter and justify the final configuration, not to sweep the full hyperparameter space.

3.4 Shared implementation infrastructure

To keep the two agents directly comparable, both scripts (`tabular_q_learning.py` and `dqn_cliffwalking.py`) share the same structure: a `Config` dataclass collecting every hyperparameter of a run, a `train()` function, a separate `evaluate()` function implementing the protocol of Section 3.2, a `plot_metrics()` function producing a standard 4-panel figure (reward per episode, success rate, episode length, exploration rate, all smoothed with a 50-episode moving average), and an `append_summary()` function that appends one row per run to a cumulative `summary.csv` file, so that all runs of an agent can be compared in a single table without re-reading individual result files.

```
@dataclass
class Config:
    learning_rate: float = 0.1
    epsilon_decay: float = 1.2
    starting_e: float = 1.0
    discount_factor: float = 0.99
    episodes: int = 5000
    max_episode_steps: int = 300
    is_slippery: bool = True
    eval_episodes: int = 100
    exp_name: str = "baseline"
    # exp_name labels the run folder and the summary.csv row
```

Listing 3.1 – Config dataclass shared by both agents (DQN adds `batch_size`, `buffer_size` and `nn_nodes`).

Exploration follows the same linear ϵ -greedy decay in both agents:

```
epsilon = max(epsilon - (epsilon_decay / episodes), 0.001)
```

applied once per episode, so `epsilon_decay` controls how many episodes it takes for ϵ to reach its floor of 0.001. Every run is executed once (single seed); no multi-seed averaging is performed, consistently across all experiments in the report.

4. Tabular Q-Learning

4.1 Implementation

The Q-table is a NumPy array of shape (48, 4), initialised to zero. For every step, an action is chosen with the ϵ -greedy rule and the corresponding entry is updated with the non-deterministic Bellman rule of Section 2.3:

```
if random.random() < epsilon:
    action = env.action_space.sample()
else:
    action = np.argmax(q_table[state, :])

new_state, reward, terminated, truncated, _ = env.step(action)

q_table[state, action] = q_table[state, action] + learning_rate * (
    reward + discount_factor * np.max(q_table[new_state, :]) - q_table[state, action]
)
```

Listing 4.1 – Action selection and Q-table update, from train() in tabular_q_learning.py.

An episode ends when the agent reaches the goal (terminated) or exceeds max_episode_steps (truncated). Falling into the cliff does not end the episode: the environment applies the -100 reward and resets the agent to the start state, and the episode continues from there until termination or truncation.

4.2 Deterministic environment: validating the update rule

Before training in the stochastic environment, two runs with is_slippery=False are used to check that the implementation reaches the known optimal return of -13. Both the baseline learning rate (0.1) and an aggressive one (0.9) converge to the optimum:

learning_rate=0.1, epsilon_decay=1.2, discount_factor=0.99, episodes=5000, is_slippery=False

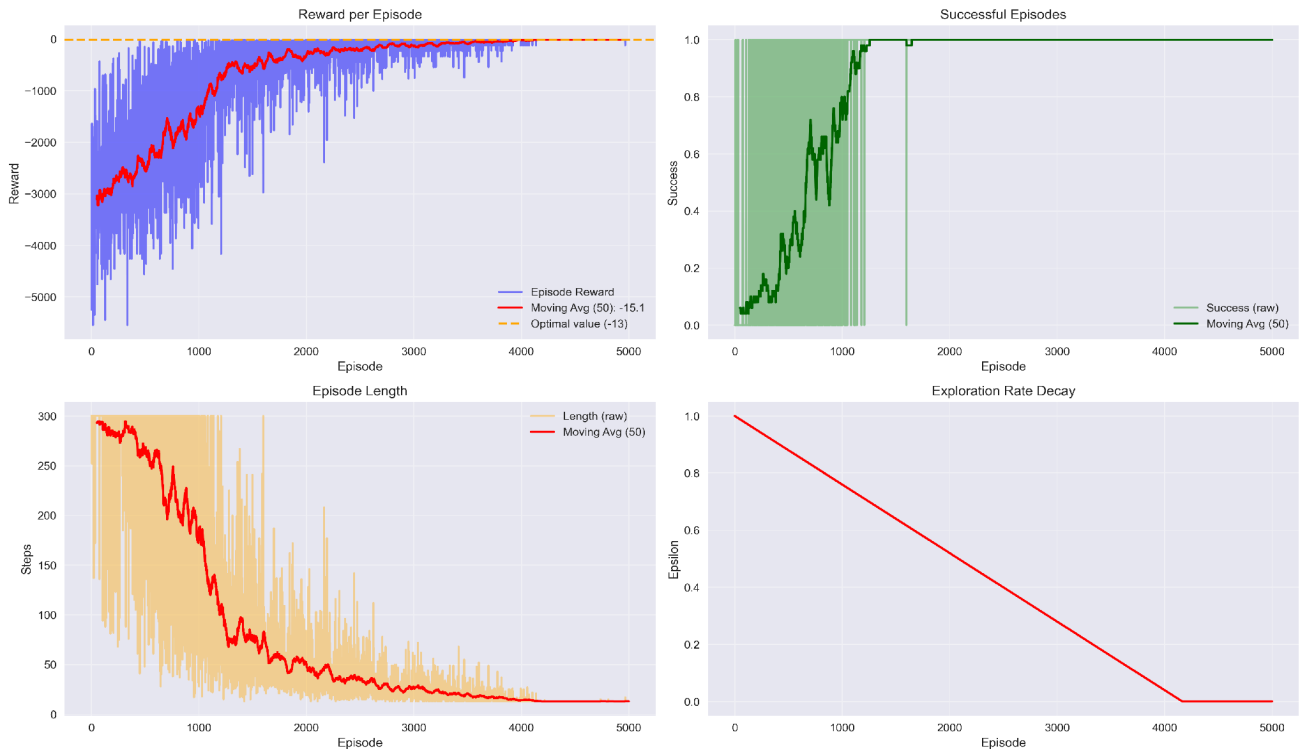


Figure 4.1 – Deterministic environment, $\text{learning_rate}=0.1$: training converges to the optimal return of -13 with 100% success.

With deterministic transitions there is no conflicting evidence for any (s,a) pair, so even a large learning rate is stable: both deterministic runs reach an evaluation return of exactly -13.00 with 100% success, matching the theoretical optimum of Section 3.1. This confirms the update rule and the training loop are implemented correctly before introducing stochasticity.

4.3 Stochastic baseline

With `is_slippery=True` and the baseline configuration (`learning_rate=0.1`, `discount_factor=0.99`, `epsilon_decay=1.2`, 5000 episodes), the agent reaches an evaluation return of -69.45 with 100% success rate.

learning_rate=0.1, epsilon_decay=1.2, discount_factor=0.99, episodes=5000, is_slippery=True

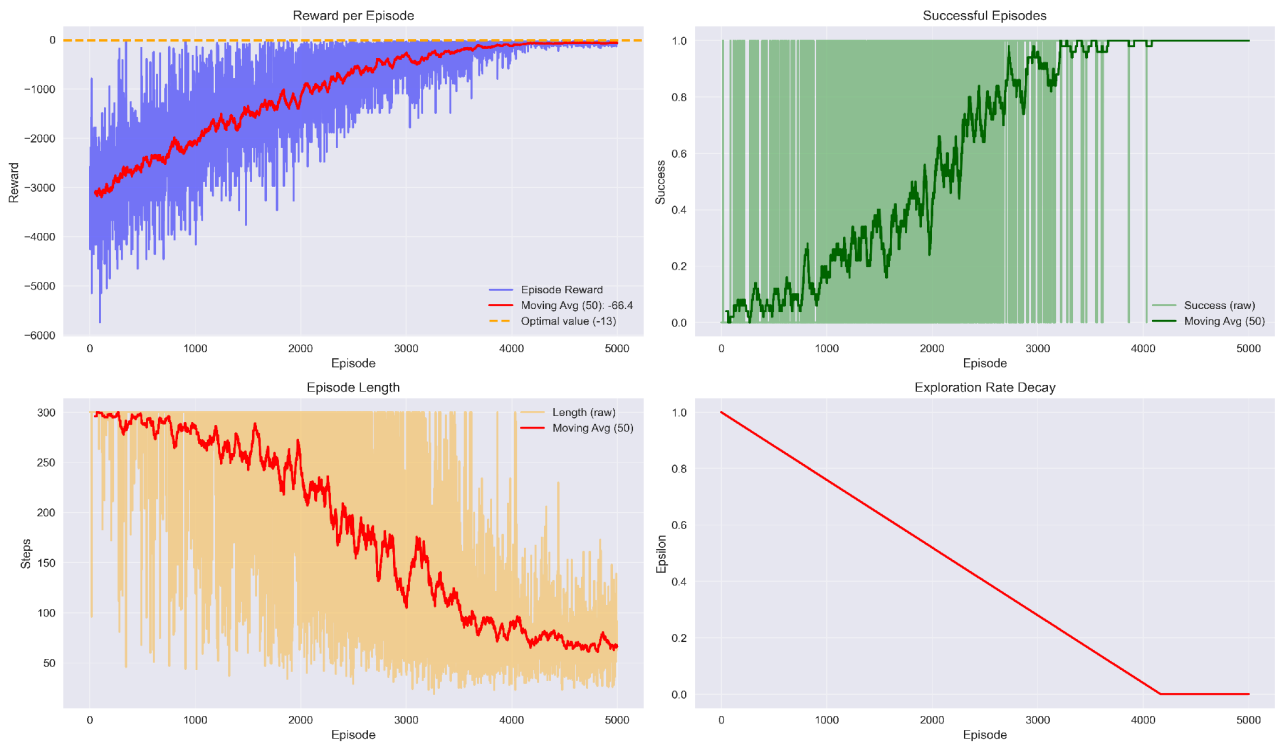


Figure 4.2 – Stochastic baseline (*is_slippery=True*): training reward stabilises around -67 , well above the deterministic optimum of -13 .

The gap between -13 (deterministic) and -69.45 (stochastic baseline) is the direct consequence of the slip probability: even a fully converged optimal policy occasionally slips toward the cliff or away from the shortest path, and every such deviation costs additional -1 steps or a -100 cliff penalty followed by a respawn. This deterministic-vs-stochastic gap is the single largest effect observed across all tabular experiments, larger than that of any individual hyperparameter change explored below.

4.4 Hyperparameter isolation

Learning rate

Starting from the baseline ($\alpha=0.1$), the learning rate is varied while keeping every other parameter fixed: $\alpha=0.01$ (*lr_low*), $\alpha=0.5$ (*lr_high*) and $\alpha=0.9$ (*lr_09*). The evaluation returns are -63.17 , -99.27 and -90.36 respectively, against -69.45 for the baseline. $\alpha=0.9$ is the only configuration in the entire tabular experiment set with an evaluation success rate below 100% (99.0%) and a training-time success rate of only 97.0%, confirming that aggressive updates make learning noisier and less stable in the stochastic environment: a single unlucky transition can overwrite a large fraction of an entry's current estimate.

learning_rate=0.9, epsilon_decay=1.2, discount_factor=0.99, episodes=5000, is_slippery=True

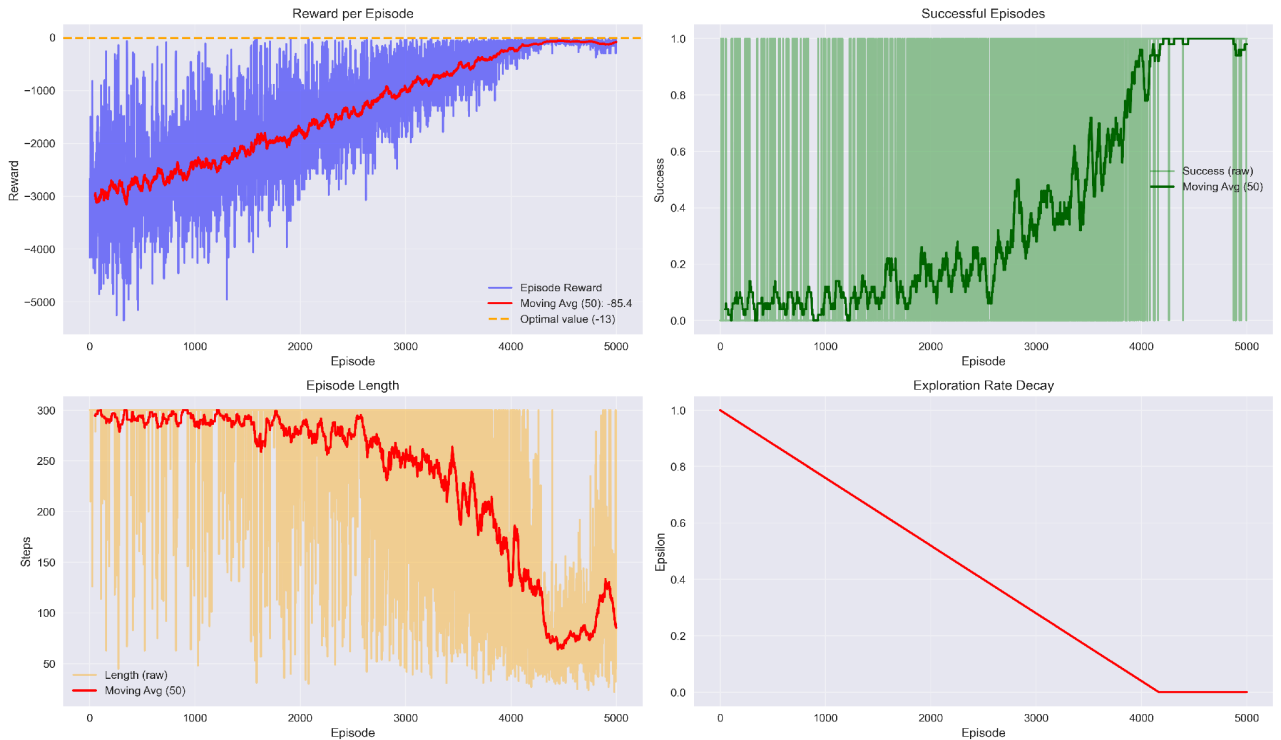


Figure 4.3 – *learning_rate=0.9: the reward curve is visibly noisier than the baseline and success rate dips even in the later episodes.*

Discount factor

The discount factor is varied around the baseline value of 0.99: $\gamma=0.90$ (gamma_low), $\gamma=0.85$ (gamma_085) and $\gamma=0.999$ (gamma_high), giving evaluation returns of -61.68 , -71.31 and -69.86 respectively. All three remain at 100% success rate. Lowering γ makes the agent value immediate reward more than long-term consequences, which on this task (where every step costs the same -1 regardless of distance to the cliff) does not visibly hurt performance in the range tested — in fact gamma_low slightly outperforms the baseline. This is different from the effect γ has on the DQN agent (Section 5.4), where lowering γ to 0.90 causes a much larger drop in performance.

Number of training episodes

With only 1000 episodes (episodes_low) the evaluation return is -78.23 , worse than the 5000-episode baseline (-69.45), while extending training to 20000 episodes (episodes_high) only marginally improves it to -64.17 , at roughly $4.4\times$ the training time (144.8 s vs 32.8 s). This indicates the tabular agent has essentially converged well before 5000 episodes, and additional training mostly refines rarely-visited state-action pairs without materially changing the evaluated policy.

Learning rate $\times$ epsilon decay interaction

The lr_high_slow_decay run combines a high learning rate ($\alpha=0.5$) with a much slower epsilon decay (epsilon_decay=0.6 instead of 1.2). With this schedule, ϵ is still above 0.4 after all 5000 episodes, so the agent keeps acting mostly at random throughout training. The

training-time success rate collapses to 50% and the training reward never approaches the optimum, even though the evaluation run (which forces $\epsilon=0$) still reaches a reasonable -117.78 with 98% success — the Q-table itself is not badly corrupted, but the persistently high exploration rate means the training metrics alone would give a misleadingly pessimistic picture if the greedy evaluation pass were not run separately.

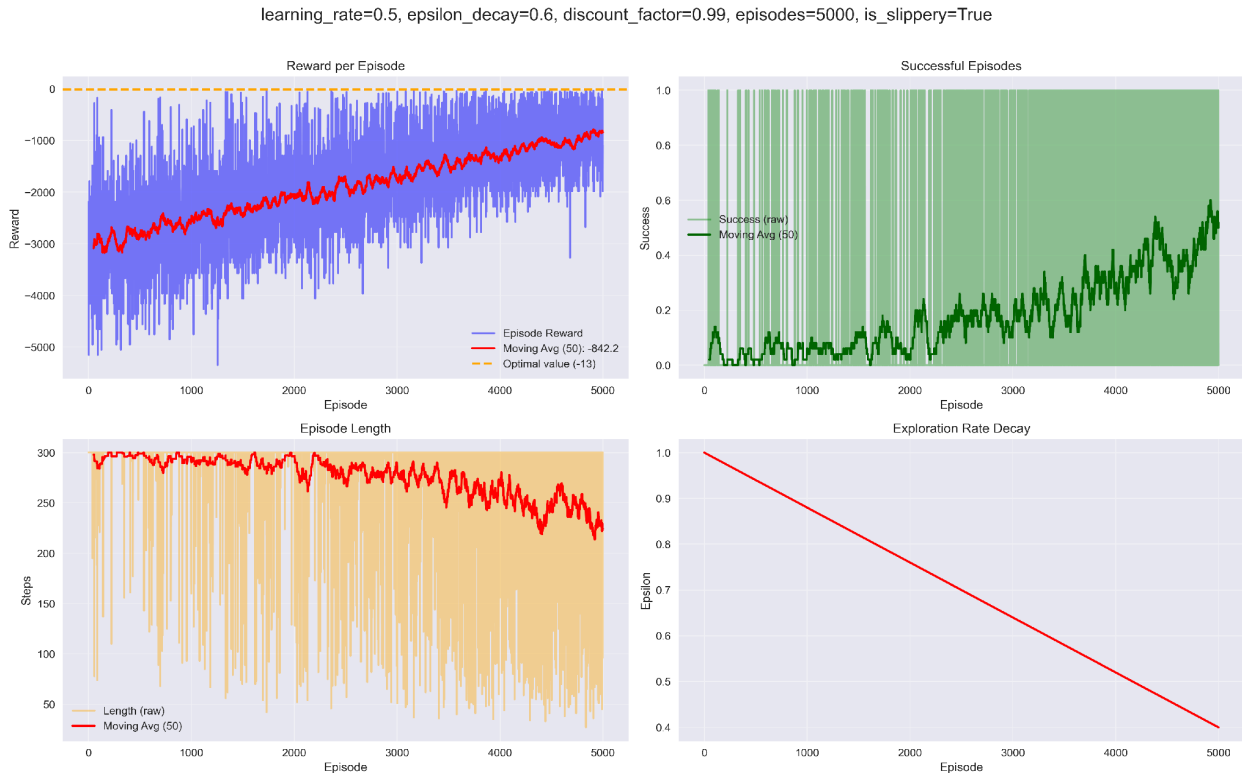


Figure 4.4 – learning_rate=0.5 with slow epsilon decay: epsilon (bottom-right) is still above 0.4 at the end of training, keeping the raw training reward far from optimal.

This run illustrates why Section 3.2 evaluates the greedy policy separately from the raw training curve: judging this configuration from the training reward alone would suggest a failed run, while the evaluation pass shows the underlying policy is in fact close to the other stochastic configurations.

4.5 Combined configurations and final choice

combo_lr_low_gamma_low combines a lower learning rate (0.01) with a lower discount factor (0.90); it reaches -73.06 , in line with the individual effects observed above rather than a clear improvement over either change alone. final_best extends training to 8000 episodes with a slightly slower epsilon decay (1.0), reaching -63.86 at a training cost of 62.3 s. final_best_v2 instead keeps the baseline's 5000 episodes and epsilon_decay=1.2, only lowering the discount factor to 0.90, and reaches -67.16 in 38.1 s — essentially the same return as final_best at little more than half the training time.

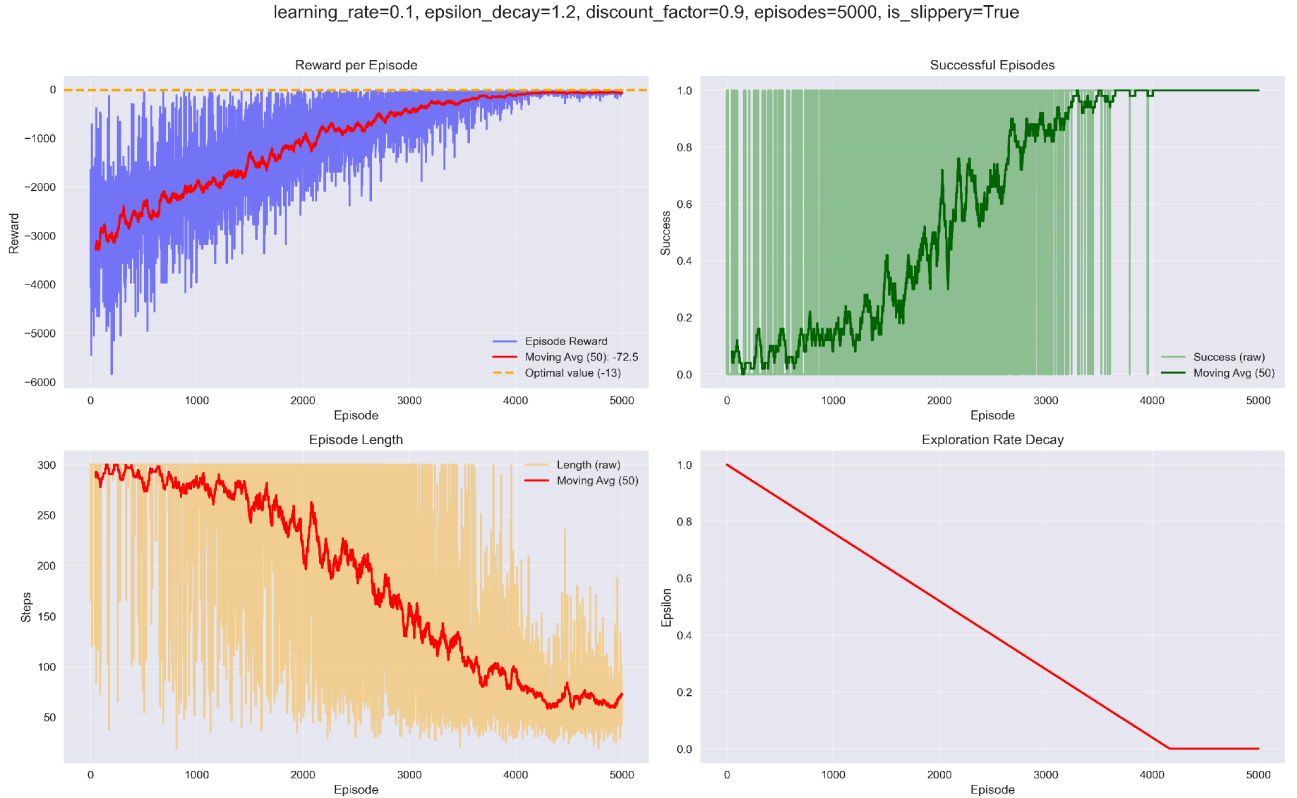


Figure 4.5 – Final tabular configuration (learning_rate=0.1, discount_factor=0.90, 5000 episodes): evaluation return -67.16, 100% success.

final_best_v2 (learning_rate=0.1, discount_factor=0.90, epsilon_decay=1.2, 5000 episodes) is taken as the final tabular configuration: it matches the best evaluation returns obtained across the whole sweep while keeping the same training budget as the baseline.

4.6 Summary

Table 4.1 collects all 16 tabular runs. eval_avg_reward and eval_success_rate are the greedy-policy metrics of Section 3.2; time_s is the wall-clock training time.

exp_name	α	ϵ -decay	γ	episodes	slippery	train $\bar{R}$	train succ.%	eval $\bar{R}$	eval succ.%	time (s)
baseline	0.1	1.2	0.99	5000	True	-66.97	100.0	-69.45	100.0	32.8
lr_low	0.01	1.2	0.99	5000	True	-76.85	100.0	-63.17	100.0	37.8
lr_high	0.5	1.2	0.99	5000	True	-79.88	100.0	-99.27	100.0	40.1
lr_09	0.9	1.2	0.99	5000	True	-101.80	97.0	-90.36	99.0	45.5
gamma_low	0.1	1.2	0.90	5000	True	-65.96	100.0	-61.68	100.0	36.5
gamma_high	0.1	1.2	0.999	5000	True	-73.77	100.0	-69.86	100.0	35.7
gamma_085	0.1	1.2	0.85	5000	True	-76.98	100.0	-71.31	100.0	36.3
episodes_low	0.1	1.2	0.99	1000	True	-75.03	100.0	-78.23	100.0	7.9
episodes_high	0.1	1.2	0.99	20000	True	-67.24	100.0	-64.17	100.0	144.8
deterministic	0.1	1.2	0.99	5000	False	-14.03	100.0	-13.00	100.0	14.7
lr09_deterministic	0.9	1.2	0.99	5000	False	-13.00	100.0	-13.00	100.0	15.2

exp_name	α	ϵ -decay	γ	episodes	slippery	train $\bar{R}$	train succ.%	eval $\bar{R}$	eval succ.%	time (s)
lr_high_slow_decay	0.5	0.6	0.99	5000	True	-867.49	50.0	-117.78	98.0	50.8
lr_low_many_episodes	0.01	1.2	0.99	20000	True	-66.22	100.0	-64.00	100.0	120.4
combo_lr_low_gamma_low	0.01	1.2	0.90	5000	True	-72.50	100.0	-73.06	100.0	35.3
final_best	0.1	1.0	0.99	8000	True	-74.92	100.0	-63.86	100.0	62.3
final_best_v2	0.1	1.2	0.90	5000	True	-69.11	100.0	-67.16	100.0	38.1

Table 4.1 – Full summary of tabular Q-learning experiments (from `Q_table/summary.csv`).

Three findings stand out from the table. First, the deterministic-vs-stochastic gap (-13 vs. ≈ -65 to -70 for most configurations) dwarfs the effect of any single hyperparameter in the stochastic setting. Second, the tabular agent is remarkably robust to hyperparameter choice: 15 of the 16 runs reach 100% success rate, and evaluation returns for reasonable settings cluster fairly tightly between -61 and -80 , with $\alpha=0.9$ and the slow-decay run as the only clear outliers. Third, training time scales almost linearly with the number of episodes and is otherwise very cheap (tens of seconds), which is the main practical difference from DQN, discussed in Section 5 and compared directly in Section 6.

5. Deep Q-Network (DQN)

5.1 Implementation

The Q-network is a small feed-forward network with two hidden layers of `nn_nodes` units each (ReLU activations), taking a one-hot encoding of the 48 discrete states as input and producing 4 Q-values, one per action:

```
class DQN(nn.Module):
    def __init__(self, n_states, nn_nodes, n_actions):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(n_states, nn_nodes), nn.ReLU(),
            nn.Linear(nn_nodes, nn_nodes), nn.ReLU(),
            nn.Linear(nn_nodes, n_actions)
        )
    def forward(self, x):
        return self.network(x)
```

Listing 5.1 – Q-network architecture, from `dqn_cliffwalking.py`.

States are converted to one-hot vectors via a precomputed identity matrix (`torch.eye(48)`), avoiding rebuilding it at every step. A `ReplayBuffer` stores transitions in a fixed-size deque, and training only starts once the buffer holds more than one full batch:

```
if len(buffer) > cfg.batch_size:
    batch = buffer.sample(cfg.batch_size)
    loss = optimize(q_network, optimizer, loss_fn, batch, identity, cfg)
```

Each gradient step (`optimize()`) computes the Bellman target with the same network under `torch.no_grad()` and takes one Adam step to minimise the MSE between predicted and target Q-values, exactly as described in Section 2.5:

```
with torch.no_grad():
    next_q = q_network(new_states_t).max(dim=1).values
    targets = rewards_t + discount_factor * next_q * (~dones_t).float()

predicted_q = q_network(states_t).gather(1, actions_t.unsqueeze(1)).squeeze(1)
loss = loss_fn(predicted_q, targets)
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

Listing 5.2 – Single-network Bellman target and gradient step, from `optimize()` in `dqn_cliffwalking.py`.

5.2 Deterministic environment

As with the tabular agent, a deterministic run (`dqn_deterministic`, 1000 episodes) is used as a sanity check. It reaches an evaluation return of exactly -13.00 with 100% success, matching the theoretical optimum and confirming the network correctly learns the Bellman target when there is no transition noise to fit around.

5.3 Stochastic baseline

With `is_slippery=True` and the baseline configuration (`learning_rate=0.001`, `discount_factor=0.99`, `batch_size=256`, `buffer_size=8192`, `nn_nodes=32`, 2500 episodes), the agent reaches an evaluation return of -69.65 with 100% success. A repeated run with the identical configuration (`dqn_stochastic_baseline`) reaches -70.25 , confirming the result is reproducible within normal run-to-run noise despite using a single seed.

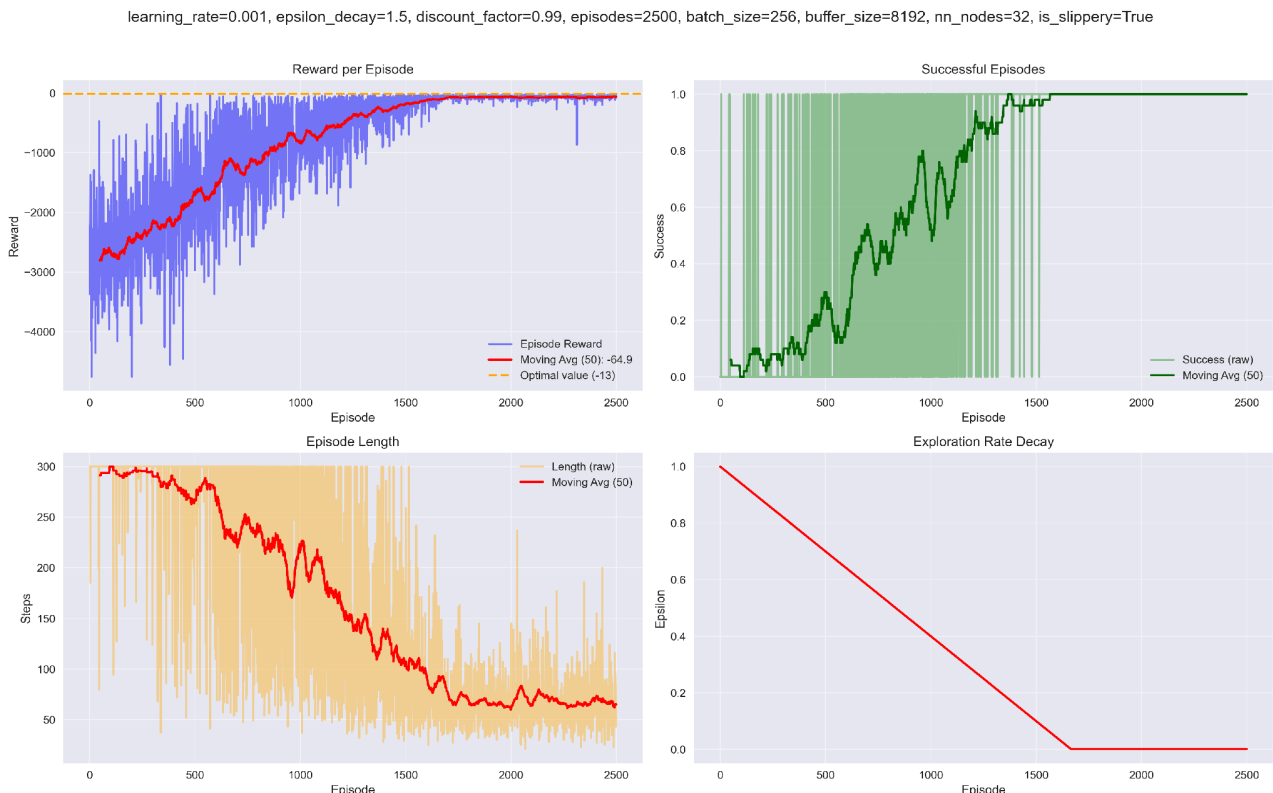


Figure 5.1 – DQN stochastic baseline: training reward converges to the same ≈ -65 to -70 region reached by tabular Q-learning.

This baseline evaluation return is essentially identical to the tabular baseline of Section 4.3 (-69.45), even though the two agents reach it through very different mechanisms (explicit table update vs. gradient descent on a sampled minibatch). Training time, however, is not comparable at all: 1545.7 s for this DQN run versus 32.8 s for the tabular baseline, a difference discussed further in Section 6.

5.4 Hyperparameter isolation

Network width

Reducing the hidden layer width from 32 to 16 units (`nodes_16`, 1500 episodes) gives an evaluation return of -64.59 with 100% success (99.0% at training time), essentially matching the wider network. Since CliffWalking-v1 only has 48 states and 4 actions, a 16-unit hidden layer already has more than enough capacity to represent the optimal Q-values for this problem; the width bottleneck one would expect on a harder task does not appear here.

Batch size

Reducing the batch size from 256 to 32 (`batch_32`) or 128 (`batch_128`) gives evaluation returns of -66.63 and -69.07 respectively, both close to the corresponding baseline at 1500 episodes. Smaller batches make individual gradient steps noisier but did not prevent convergence in these runs; batch size had a visibly smaller effect than the replay-buffer size discussed next.

Replay buffer size: a collapse case

Shrinking the replay buffer from 8192 to 256 (`buffer_256`) causes a dramatic collapse: evaluation return -2058.34 with only 85% success, against -64.49 / -67.66 for the two `buffer_4096` repeats and -69.65 for the full baseline.

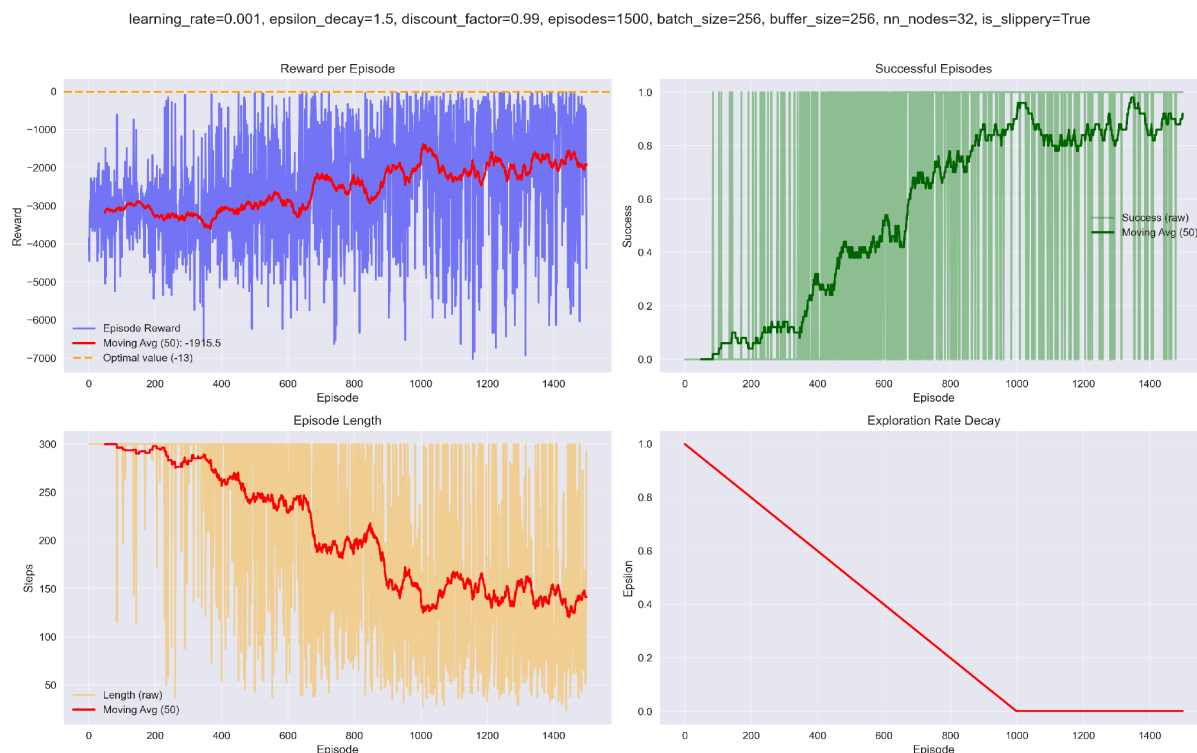


Figure 5.2 – `buffer_size=256`: reward stays far below the optimum and remains extremely noisy even after ϵ reaches 0 around episode 1000.

The cause of this collapse is a direct consequence of `buffer_size` being set equal to `batch_size` (both 256). The replay buffer is a deque with `maxlen=buffer_size`, so its length can never exceed 256; the training condition `len(buffer) > cfg.batch_size` (i.e. `len(buffer) > 256`) is therefore never satisfied, and `optimize()` is never called for the entire run. This is confirmed directly by the logged results: the average training loss for this run is 0.0000, meaning no gradient steps were ever taken. The network stays at its random initialisation throughout training, so once ϵ decays toward zero the agent is effectively acting greedily with respect to an untrained, arbitrary set of Q-values — which explains both the very poor average reward and the fact that the episode-length panel in Figure 5.2 stays high and noisy instead of shrinking toward the optimal path length. This is the single most informative failure case among all the experiments in this report, since it isolates a pure implementation/configuration bug (`buffer_size ≤ batch_size`) rather than a genuine learning-dynamics effect.

With `buffer_size=4096` (still smaller than the baseline's 8192, but comfortably larger than `batch_size`), training proceeds normally and reaches -64.49 / -67.66 across the two repeats, confirming that the collapse above is specifically about the buffer/batch-size relationship rather than buffer size in general being a sensitive parameter.

Discount factor

Lowering the discount factor from 0.99 to 0.90 (`gamma_0.90`, otherwise identical to the baseline) gives an evaluation return of -138.30 with only 94% success, clearly worse than the -69.65 baseline. This contrasts with the tabular agent (Section 4.4), where lowering γ to

0.90 barely changed performance (-67.16 vs. -69.45 baseline). A plausible explanation is that a lower γ shrinks the magnitude of the bootstrapped target $r + \gamma \max Q(s', a')$, which changes the scale of the regression target the network has to fit; the tabular update simply stores whatever value results with no such fitting process, so it is less sensitive to this rescaling.

Epsilon decay

`epsilon_decay` controls how quickly exploration is phased out. A slower decay (`epsilon_decay=1.0`, `eps_1.0`) leaves the network training on a more exploration-heavy stream of transitions and reaches -93.69 , worse than the baseline's -69.65 . A faster decay (`epsilon_decay=2.0`, `eps_2.0`) reaches -61.90 with 100% success — the best evaluation return obtained by the DQN agent across every experiment in this report.

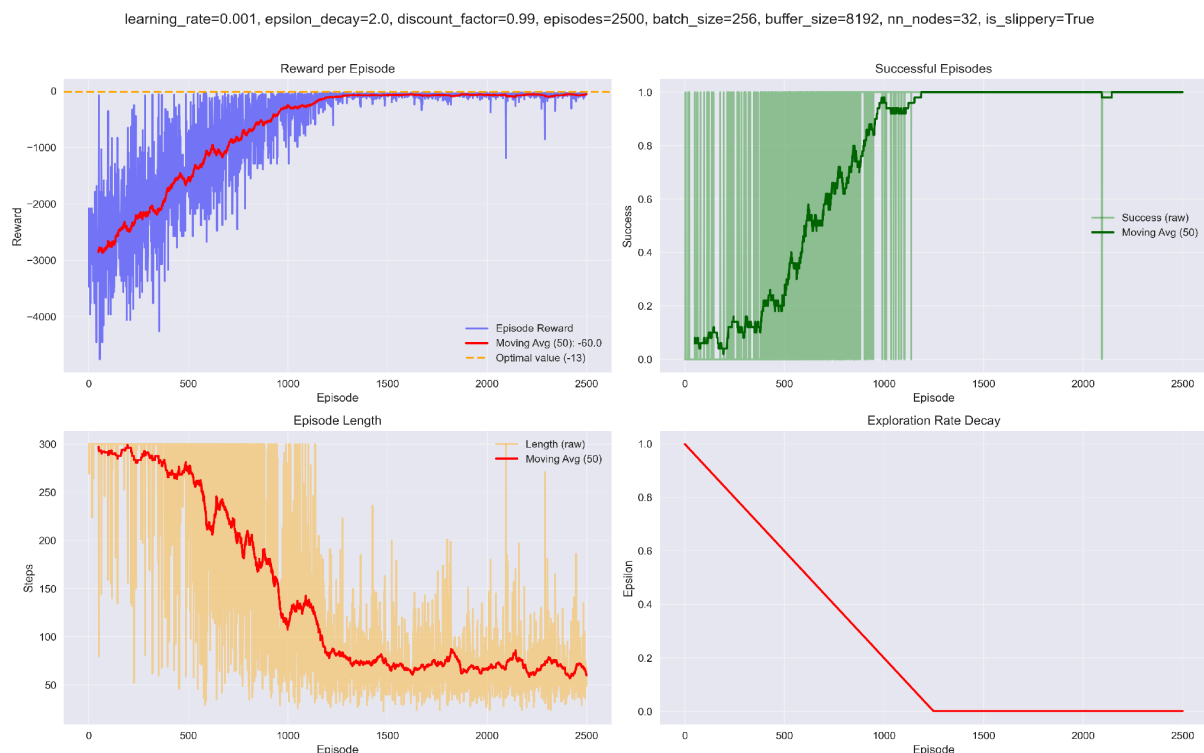


Figure 5.3 – `epsilon_decay=2.0`, the best-performing DQN configuration: evaluation return -61.90 , 100% success.

This suggests that, once the network has seen enough transitions to fit a reasonable Q function, further exploration mostly adds noisy, off-policy transitions to the replay buffer without adding much new information, so cutting exploration short is beneficial here.

Learning rate

Raising the learning rate from 0.001 to 0.01 (`lr_0.01`) gives -75.60 , while lowering it to 0.0001 (`lr_0.0001`) gives -86.13 , both worse than the 0.001 baseline (-69.65). Unlike the tabular agent, where a very small learning rate (0.01) still reached a good result (-63.17 , Section 4.4), DQN's gradient-based updates are comparatively sensitive to the learning rate

in both directions: too high destabilises the fit, too low fails to converge within the fixed 2500-episode budget.

5.5 Training loss

Figure 5.4 shows the MSE training loss for the baseline run. The loss starts high (reflecting the random initial network and its large initial disagreement with the Bellman target), drops sharply within the first ≈ 100 episodes as the network fits the coarse shape of the value function, and then decreases gradually as ϵ anneals and the target values stabilise, flattening out once ϵ reaches its floor around episode 2000.

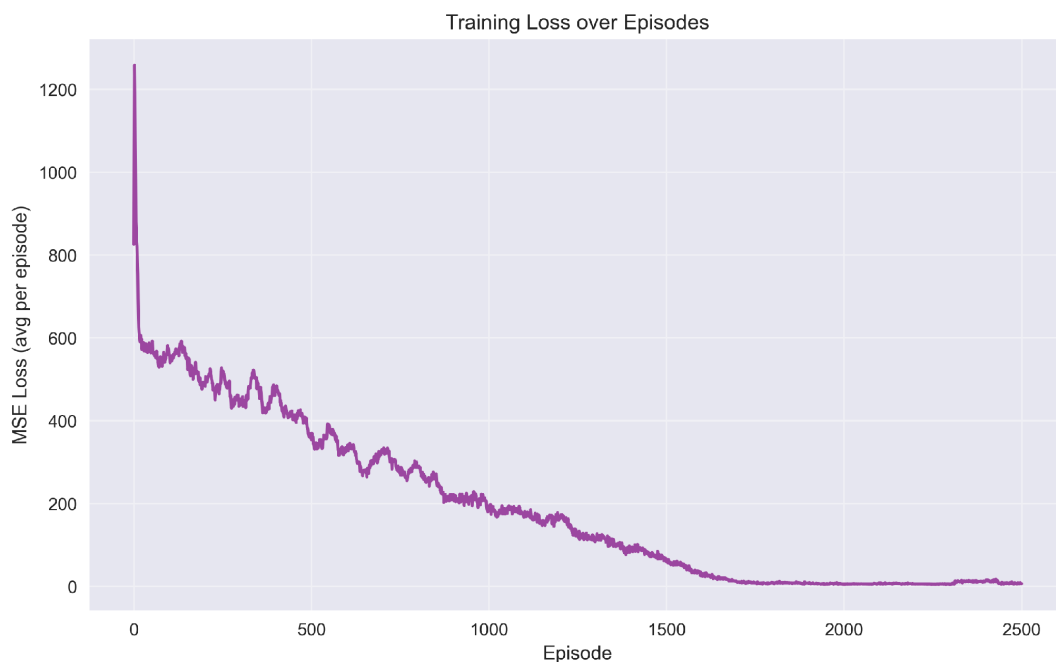


Figure 5.4 – DQN training loss (baseline run): sharp early drop followed by gradual convergence as exploration anneals.

This loss curve is a useful complement to the reward curve: a healthy run shows loss decreasing broadly in step with reward increasing, while the `buffer_256` run of Section 5.4 (constant zero loss because no optimisation step is ever executed) is the clearest possible counter-example, immediately visible if the two curves are plotted side by side.

5.6 Final configuration

Combining the findings above — network width and batch size have limited effect on this small problem, a large-enough replay buffer is required (`buffer_size` $\gg$ `batch_size`), `discount_factor=0.99` outperforms 0.90, and a faster epsilon decay outperforms both the baseline decay and a slower one — `eps_2.0` (`learning_rate=0.001`, `discount_factor=0.99`, `epsilon_decay=2.0`, `batch_size=256`, `buffer_size=8192`, `nn_nodes=32`, 2500 episodes) is taken as the final DQN configuration, with an evaluation return of -61.90 and 100% success rate.

5.7 Summary

Table 5.1 collects all 15 DQN runs, in the same format as Table 4.1.

exp_name	lr	ϵ -dec	γ	epis.	slip.	batch	buffer	nodes	eval $\bar{R}$	eval succ. %	time (s)
baseline	0.001	1.5	0.99	2500	True	256	8192	32	-69.65	100.0	1545.7
ep_1000	0.001	1.5	0.99	1000	True	256	8192	32	-69.50	100.0	593.3
dqn_deterministic	0.001	1.5	0.99	1000	False	256	8192	32	-13.00	100.0	256.9
nodes_16	0.001	1.5	0.99	1500	True	256	8192	16	-64.59	100.0	844.3
batch_32	0.001	1.5	0.99	1500	True	32	8192	32	-66.63	100.0	757.5
batch_128	0.001	1.5	0.99	1500	True	128	8192	32	-69.07	100.0	775.8
buffer_256	0.001	1.5	0.99	1500	True	256	256	32	-2058.34	85.0	49.9
buffer_4096 (run 1)	0.001	1.5	0.99	1500	True	256	4096	32	-64.49	100.0	913.5
buffer_4096 (run 2)	0.001	1.5	0.99	1500	True	256	4096	32	-67.66	100.0	848.7
gamma_0.90	0.001	1.5	0.90	2500	True	256	8192	32	-138.30	94.0	1678.9
eps_1.0	0.001	1.0	0.99	2500	True	256	8192	32	-93.69	100.0	1840.8
eps_2.0	0.001	2.0	0.99	2500	True	256	8192	32	-61.90	100.0	1256.3
lr_0.01	0.01	1.5	0.99	2500	True	256	8192	32	-75.60	100.0	1979.1
lr_0.0001	0.0001	1.5	0.99	2500	True	256	8192	32	-86.13	100.0	1428.1
dqn_stochastic_baseline	0.001	1.5	0.99	2500	True	256	8192	32	-70.25	100.0	1413.6

Table 5.1 – Summary of DQN experiments, evaluation metrics only (from DQN/summary.csv; full training-time columns omitted for space).

The overall pattern for DQN is different from the tabular agent in one important respect: while most configurations still land in the same ≈ -62 to -86 range as the tabular sweep, DQN produces one catastrophic outlier (buffer_256) that has no tabular equivalent, because the failure mode — a network that never receives a single gradient update — is specific to the replay-buffer/minibatch mechanism and does not exist in the tabular setting, where every single transition updates the table immediately. DQN is also markedly more sensitive to the discount factor and to learning rate than the tabular agent, consistent with the fact that it is fitting a regression target rather than directly overwriting a table entry. Training time is the other major difference: DQN runs take from tens of minutes to over half an hour on CPU, roughly two to three orders of magnitude slower than the equivalent tabular run — compared directly in Section 6.

6. Comparative Analysis: Tabular Q-Learning vs DQN

This section compares the two agents directly, using the deterministic sanity check, the stochastic baseline, and each agent's final configuration from Sections 4 and 5.

6.1 Evaluation return

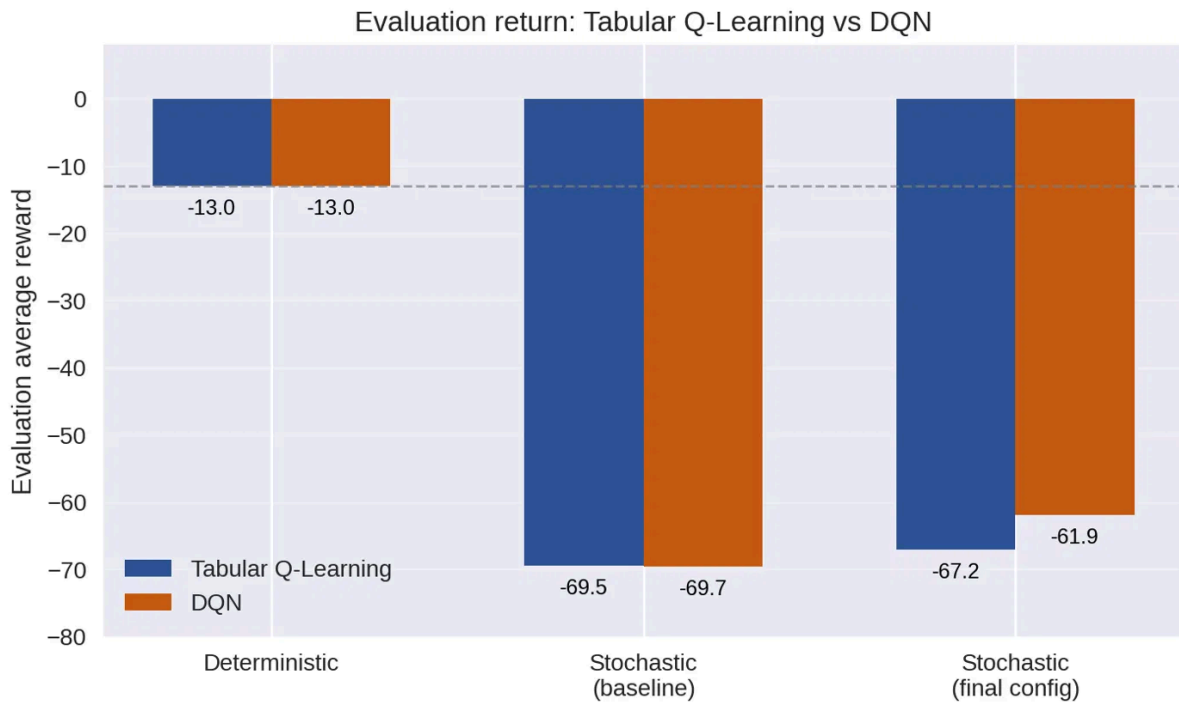


Figure 6.1 – Evaluation-time average reward, Tabular Q-Learning vs DQN, across matched configurations.

Both agents recover the exact theoretical optimum of -13 in the deterministic environment, and land within a couple of points of each other on the stochastic baseline (-69.45 vs. -69.65). After tuning, DQN's final configuration (-61.90) slightly outperforms the tabular agent's (-67.16), but the difference is small compared to the gap either agent shows between its worst and best stochastic configuration. In other words, for a problem as small as CliffWalking-v1, representation (table vs. network) has less impact on the final achievable return than getting the hyperparameters right within each representation.

6.2 Training time

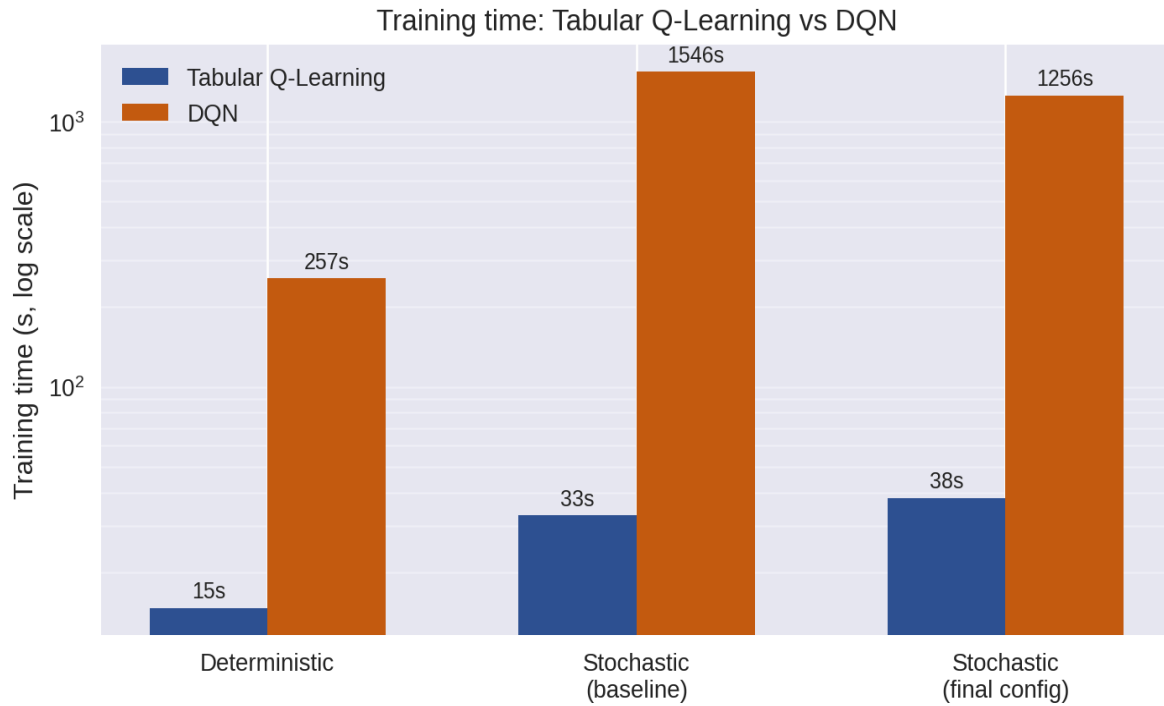


Figure 6.2 – Training time (log scale), Tabular Q-Learning vs DQN.

Training time tells a very different story. Across every matched configuration, DQN takes roughly 17–40× longer to train than the tabular agent on the same number of episodes (e.g. 1545.7 s vs. 32.8 s on the stochastic baseline). This is the clearest, most consistent difference between the two agents observed in this project: the tabular update is an $O(1)$ array write per step, while DQN performs a full forward and backward pass through a neural network on a 256-sample minibatch at (potentially) every step once the replay buffer is warmed up. For a state space as small as CliffWalking-v1's 48 states, this cost is not repaid by any corresponding gain in final performance — the tabular agent reaches essentially the same return in a fraction of the time.

6.3 Stability and failure modes

Table 6.1 places the worst run observed for each agent side by side with the matched configurations of Sections 6.1–6.2.

Configuration	Tabular (eval $\bar{R}$ / succ.)	DQN (eval $\bar{R}$ / succ.)	Tabular time	DQN time
Deterministic	−13.00 / 100%	−13.00 / 100%	14.7 s	256.9 s
Stochastic – baseline config	−69.45 / 100%	−69.65 / 100%	32.8 s	1545.7 s
Stochastic – final config	−67.16 / 100%	−61.90 / 100%	38.1 s	1256.3 s
Worst observed run	−90.36 / 99% ($\alpha=0.9$)	−2058.34 / 85% (buffer=256)	45.5 s	49.9 s

Table 6.1 – Side-by-side comparison of key configurations.

Both agents are vulnerable to badly chosen hyperparameters, but the failure modes are qualitatively different. The tabular agent's worst case (`learning_rate=0.9`) degrades gracefully: the evaluation success rate only drops to 99% and the return roughly doubles in magnitude, because every update, however aggressive, is still an average pulling the table entry toward the observed target. DQN's worst case (`buffer_size=256`) is a hard failure with no equivalent in the tabular setting: because the replay buffer never grows past the batch size, the network receives literally zero gradient updates for the entire run and the agent acts on an untrained network once exploration anneals, producing a return over 30× worse than the baseline. This type of failure — a silent no-op in the training loop rather than a slow or unstable optimisation — is specific to the added machinery (replay buffer, minibatching) that function approximation requires and is a useful concrete illustration of why DQN has more moving parts to get right than tabular Q-learning.

6.4 Sample efficiency

A closer look at the episode counts needed to approach the optimum reinforces the same picture. In the tabular case, comparable performance is reached anywhere from 1000 episodes (`episodes_low`, -78.23) to 20000 episodes (`episodes_high`, -64.17), with diminishing returns well before 5000 episodes. DQN's best result (`eps_2.0`) is obtained with 2500 episodes but needs a carefully tuned exploration schedule to get there — the default exploration schedule (`epsilon_decay=1.5`, the baseline) needs the same 2500 episodes to reach a visibly worse return. This suggests DQN is not more sample-efficient than the tabular agent on this task; if anything it is comparably sample-hungry while being far more expensive per sample.

6.5 Summary of the comparison

- Final return: comparable, with DQN's tuned configuration slightly ahead (-61.90 vs. -67.16), well within the spread either agent shows across its own hyperparameter sweep.
- Training time: tabular Q-learning is 17–40× faster on identical episode budgets, and this gap does not close as hyperparameters are tuned.
- Stability: the tabular agent degrades gracefully under poor hyperparameters; DQN can fail completely and silently (`buffer_size ≤ batch_size`) in a way with no tabular analogue.
- Sample efficiency: comparable number of episodes needed to reach a good policy on this small state space, so DQN's function-approximation machinery does not pay for itself here — its advantage would be expected to appear on problems where an explicit table is infeasible, not on a 48-state gridworld.

7. Conclusion

7.1 Summary of findings

Both Tabular Q-Learning and DQN were implemented from scratch and trained on CliffWalking-v1 using the non-deterministic Bellman update rule required by the project. Both

agents recover the exact theoretical optimum (-13) in the deterministic environment, confirming the correctness of the update rule and of the two implementations. In the stochastic environment, both agents converge to a comparable evaluation return: -67.16 for the best tabular configuration and -61.90 for the best DQN configuration, both with 100% success rate.

The final tabular configuration is `learning_rate=0.1`, `discount_factor=0.90`, `epsilon_decay=1.2`, 5000 episodes. The final DQN configuration is `learning_rate=0.001`, `discount_factor=0.99`, `epsilon_decay=2.0`, `batch_size=256`, `buffer_size=8192`, `nn_nodes=32`, 2500 episodes. Across 16 tabular and 15 DQN experiments, the single largest effect on performance for either agent was the deterministic-vs-stochastic environment setting, not any individual hyperparameter.

The most informative single experiment in this project was `buffer_256` (Section 5.4): with `buffer_size` equal to `batch_size`, the DQN training loop never executes a single gradient step, and the agent ends up acting on an untrained network. This failure has no equivalent in the tabular agent, where every transition updates the table immediately regardless of any buffering, and it is the clearest illustration in the whole report of a risk that is specific to function approximation with experience replay.

7.2 Practical takeaways

- For a state space small enough to tabulate, Tabular Q-Learning is preferable in practice: it reaches the same return as DQN, is much less sensitive to hyperparameters, and trains faster.
- DQN's main structural risk is the interaction between `buffer_size` and `batch_size`: the buffer must be comfortably larger than the batch size, or training silently does nothing.
- The discount factor and learning rate matter more for DQN than for the tabular agent, because they affect a regression target that the network has to fit, not just a single table entry.
- Evaluating the greedy policy separately from the training-time metrics (Section 3.2) is essential: the `lr_high_slow_decay` tabular run (Section 4.4) would look like a failed run from its training curve alone, while its evaluation return is close to the other stochastic configurations.